MACHINE LEARNING-BASED LATE DELIVERY RISK PREDICTION

Project Report

A predictive analytics prototype for supply chain delivery-risk assessment

Technology: Python, Pandas, NumPy, Scikit-learn, Streamlit, Plotly and Joblib

Project Repository: github.com/yash-bhusari/late-delivery-risk-prediction

Prepared as an academic/internship project report

Abstract

Late deliveries can affect customer satisfaction, operational planning and supply chain performance. This project develops a machine learning prototype to estimate whether an order may be classified as late using order, product, customer, geographical and shipping-related attributes. The workflow includes data inspection, preprocessing, feature selection, model training, evaluation and an interactive Streamlit dashboard. Logistic Regression achieved an accuracy of 71.41% and a ROC-AUC score of 0.7823 on a stratified hold-out test set. A balanced Random Forest achieved 69.89% accuracy and 0.7603 ROC-AUC. Logistic Regression was therefore selected as the best overall model among the evaluated approaches. The dashboard accepts order details and displays the model's estimated late-delivery probability. The results demonstrate a practical predictive analytics workflow, while also highlighting the need for temporal validation and testing on operational data before real-world deployment.

1. Introduction

Delivery reliability is an important component of supply chain operations. Late deliveries can create downstream disruption, increase customer dissatisfaction and make planning more difficult. Predictive analytics can help identify orders that resemble historically late orders, allowing teams to investigate potential risks.

The objective of this project is to build a supervised binary classification prototype that estimates late-delivery risk from selected order attributes. The project is framed around a logistics case-study context. The dataset used is a public logistics dataset commonly associated with the DataCo Supply Chain dataset; it has not been verified as proprietary APL Logistics operational data.

2. Objectives

- Inspect and prepare a logistics dataset for supervised learning.
- Select input attributes while excluding the target, identifiers and fields considered potentially leakage-prone.
- Train and compare Logistic Regression and Random Forest classifiers.
- Evaluate model performance using accuracy, classification metrics, confusion matrices and ROC-AUC.
- Build an interactive Streamlit application to demonstrate single-order predictions.

3. Dataset Description

The dataset contains 180,519 records and 40 columns in its original form. The target variable is Late_delivery_risk, where 0 represents an order not classified as late and 1 represents an order classified as late. The observed target counts were 81,542 records in class 0 and 98,977 records in class 1.

The modelling workflow used 22 input features after excluding the target, customer identifiers and names, street and zipcode fields, status/shipping-duration fields, and selected financial fields. The selected features include order type, category, customer and order geography, customer segment, department, latitude and longitude, discount information, quantity, sales, item total, product name and price, market, region and shipping mode.

Two missing-value fields were handled during initial preparation: missing Customer Lname values were filled with 'Unknown' and missing Customer Zipcode values were filled with 0. Customer identifiers and personal-name fields were excluded from model inputs.

4. Data Preprocessing

- The data was separated into features (X) and target (y).
- A stratified train-test split was used, with 80% of records for training and 20% for testing, using random_state=42.

- Numerical columns were imputed using the median and standardized using StandardScaler.
- Categorical columns were imputed using the most frequent value and transformed using OneHotEncoder with handle_unknown='ignore'.
- A ColumnTransformer combined the numerical and categorical preprocessing pipelines.
- The transformed training data contained 5,659 encoded features.

The dataset does not contain an order-date field suitable for chronological splitting. Consequently, the project used a stratified random split rather than a time-based validation strategy.

5. Model Development

5.1 Logistic Regression

Logistic Regression was trained as a binary classification model using the preprocessed training data, with max_iter=1000 and random_state=42. It produced the strongest overall results among the tested models.

5.2 Random Forest

A basic Random Forest configuration was tested with 30 trees, a maximum depth of 15 and a minimum leaf size of 5. A second balanced Random Forest was trained with 50 trees, maximum depth 20, minimum leaf size 2 and class_weight='balanced'. The basic model strongly favoured class 1, while the balanced model produced a more balanced distribution of predictions.

6. Model Evaluation

The following results were recorded on the hold-out test set:

Model	Accuracy	ROC-AUC	Late-class precision	Late-class recall
Logistic Regression	71.41%	0.7823	0.79	0.66
Random Forest (balanced)	69.89%	0.7603	0.83	0.57
Random Forest (basic)	55.99%	0.7530	0.56	0.99

Logistic Regression was selected as the leading model because it achieved the highest accuracy and ROC-AUC among the evaluated models. The balanced Random Forest had higher precision for the late class (0.83), but lower recall (0.57). This trade-off means it identified fewer actual late orders than Logistic Regression at the model's default decision threshold.

For Logistic Regression, the recorded confusion matrix was [[12,791, 3,517], [6,804, 12,992]], with rows representing actual classes 0 and 1 and columns representing predicted classes 0 and 1.

7. Streamlit Application

A Streamlit dashboard was developed to demonstrate the trained model. The application loads the saved preprocessing pipeline and Logistic Regression model from the models directory. Users can enter order and shipping details through a form and submit them for prediction.

- Order input form covering the 22 model features.
- Predicted class indicating whether the order is classified as at risk of late delivery.
- Estimated late-delivery probability displayed as a percentage.
- Plotly gauge visualization for the probability.

- Model performance metrics and a comparison chart.
- Project overview and limitations section.

The dashboard is a demonstration interface. The probability is a model estimate based on the training data and should not be interpreted as a guaranteed delivery outcome.

8. Tools and Technologies

- Python 3.13 for implementation.
- Pandas and NumPy for data handling.
- Scikit-learn for preprocessing, model training and evaluation.
- Matplotlib and Seaborn for exploratory data analysis.
- Joblib for saving and loading trained objects.
- Streamlit for the interactive web dashboard.
- Plotly for the risk probability gauge.
- Git and GitHub for version control and project hosting.

9. Limitations

- The dataset is public and has not been verified as actual APL Logistics operational data.
- No suitable order-date field was available, so chronological validation could not be performed.
- A random train-test split may not reflect performance on future orders.
- Some selected fields may not be available at the exact time an operational prediction would be made; feature availability should be confirmed before deployment.
- One-hot encoding can produce many dimensions, and unseen categories are ignored by the configured encoder.
- Model performance is moderate and further validation, calibration and threshold analysis are required.
- The dashboard is a prototype and is not connected to live logistics systems.

10. Future Scope

- Validate the approach using time-stamped operational data and chronological train/test splits.
- Confirm that every feature is available at prediction time and remove any remaining leakage risks.
- Test additional algorithms and tune hyperparameters using cross-validation.
- Study precision-recall trade-offs and choose a decision threshold based on operational costs.
- Add model interpretation, such as permutation importance or SHAP, where appropriate.
- Deploy behind suitable access controls and monitor model drift if real operational use is considered.

11. Conclusion

This project demonstrates an end-to-end machine learning workflow for estimating late-delivery risk from logistics-related attributes. Logistic Regression achieved 71.41% test accuracy and a ROC-AUC score of 0.7823, outperforming the evaluated Random Forest configurations on these overall metrics. The Streamlit dashboard

provides a practical interface for demonstrating individual predictions and model performance. The prototype is suitable as an educational and portfolio demonstration, but operational use would require reliable time-stamped data, feature-availability checks, chronological validation and ongoing monitoring.

Project Repository

https://github.com/yash-bhusari/late-delivery-risk-prediction

Note: This report reflects the implementation details and results recorded during project development. It should be updated if the code, dataset, model results or deployment approach changes.